



# Coinage: Minting for Verified Com- pute, and What Adopting the Protocol Gives You

Zach Kelling

*Hanzo AI*

2026

## Abstract

A token sold to investors and a token minted for work performed are different instruments that happen to share a data structure. The first is bought with money in expectation of profit from someone else's effort. The second is received as compensation for a service the recipient rendered, which is a different transaction in every respect that matters.

That distinction is the most valuable asset a compute network has, and it is easy to destroy by accident. Attaching a portfolio of assets to a mined token — making it a claim on something — converts compensation back into an investment and gives away the position for nothing, because the same price support is available from the opposite arrangement at no cost.

We set out the design that preserves it: mint for verified compute, let a fund hold the token rather than back it, and keep the treasury demonstrable rather than redeemable. We then describe what a network adopting this protocol actually receives — a mint conditioned on verification rather than assertion, confidential execution with local attestation across four hardware tiers, a post-quantum retrofit that requires no fork, and a regime-aware market maker for heterogeneous capacity — all of it already implemented and permissively licensed.

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Two tokens with the same shape</b>                    | <b>3</b> |
| <b>2</b> | <b>The arrow points the other way</b>                    | <b>3</b> |
| <b>3</b> | <b>What the protocol mints for</b>                       | <b>3</b> |
| <b>4</b> | <b>What adoption gives a network</b>                     | <b>4</b> |
| 4.1      | A mint conditioned on verification . . . . .             | 4        |
| 4.2      | Confidentiality as a capability, not a promise . . . . . | 4        |
| 4.3      | A post-quantum retrofit with no fork . . . . .           | 5        |
| 4.4      | A market maker for heterogeneous capacity . . . . .      | 5        |
| <b>5</b> | <b>The composition</b>                                   | <b>5</b> |
| <b>6</b> | <b>What this does not settle</b>                         | <b>5</b> |

## 1 Two tokens with the same shape

Consider two ways a token reaches a holder.

In the first, the holder sends money to an issuer and receives tokens, expecting their value to rise through the issuer’s continuing effort. Every element of the classic securities analysis is present: money changes hands, it goes into a common enterprise, and the expectation of return rests on someone else’s work.

In the second, the holder performs a service the network requested — runs a computation, returns a verified result — and the protocol mints in payment. No money was invested. The return is not an expectation but a settled fee. The counterparty is a rule, not a promoter.

These are not variations on a theme. The second is how mining has always worked, and it is the strongest position a token can occupy. The design question is therefore not how to construct that position but how to avoid discarding it.

**Proposition 1** (Backing converts compensation into investment). *If a token’s value derives from a pool of assets the issuer controls, holders hold a claim on that pool. A claim on a managed pool is an investment interest regardless of how the holder acquired it. Minting a claim does not stop it being a claim.*

The consequence is precise: *a mined token must not be backed*. Whatever credibility backing was meant to supply has to come from somewhere else.

## 2 The arrow points the other way

The good news is that the objective behind “back the token with the fund” is fully achievable by reversing the relationship, at no legal cost.

|                       | Fund backs the token      | Fund holds the token        |
|-----------------------|---------------------------|-----------------------------|
| What the holder owns  | a claim on the fund       | a network asset             |
| Token becomes         | an investment interest    | unchanged                   |
| Mining posture        | lost                      | preserved                   |
| Affiliate transaction | prohibited without relief | ordinary portfolio decision |
| Price effect          | promised                  | actual                      |
| Composability         | restricted                | unrestricted                |

Table 1: The same two entities, related in the two possible directions.

A fund whose thesis is artificial intelligence holding an artificial-intelligence network’s asset is an ordinary portfolio decision requiring no explanation. It produces real demand rather than a promise of support, it survives diligence, and it leaves the token exactly as unencumbered as it was. The reverse arrangement produces a promise, a claim, a prohibited affiliate transaction on any registered vehicle, and a token that can no longer be liquidity for anything.

Demonstrable assets, where they are wanted, come from a foundation treasury at published addresses — verifiable without being redeemable. The distinction is the whole structure: publishing what you hold is a fact, promising that a token is backed by it is an undertaking.

## 3 What the protocol mints for

Minting for work is only as good as the definition of work, and this is where most compute networks are weakest. If the protocol mints on assertion — a provider claims to have computed something and is believed — then it is not paying for compute, it is paying for claims, and it will get the claims at the lowest possible cost.

Our position is that the mint must be conditioned on verification, and that verification has to be cheap enough to run on every job rather than a sample. Three regimes cover the field.

**Bit-reproducible work.** Where a computation is deterministic, the result is checkable against a canonical implementation at negligible cost, and a mismatch is a proof of misbehaviour rather than a vote. This covers all cryptographic work, all quantized inference with pinned numerics, all search, and all embedding generation — a larger share of real demand than is generally assumed, and a share that can be enlarged deliberately by choosing reproducible kernels.

**General execution.** Where reproducibility is impractical, confidential computing attests that the declared computation ran on declared hardware with declared inputs. Overhead for compute-bound large-model work is a few percent, because arithmetic dominates the boundary crossings, and current hardware with encrypted device links narrows what remains. We classify providers across four tiers, from accelerator-native confidential mode down to no hardware support where stake substitutes, with all attestation performed locally rather than against a vendor’s cloud — a network that cannot verify without an external dependency has not removed the trusted party.

**Closed models.** Where the computation happens inside a vendor’s closed model, nothing algebraic helps and no attestation is offered. What remains verifiable is provenance: a session terminating at the vendor’s endpoint under a pinned certificate. That guarantee belongs to whoever terminates the session, so a relay must sit inside an attested boundary for it to mean anything to a buyer.

A protocol that mints against these three regimes is paying for compute. One that mints against assertion is running an honour system with a token attached.

## 4 What adoption gives a network

A network adopting this protocol receives four things, all implemented and permissively licensed, and each independently useful.

### 4.1 A mint conditioned on verification

The determinism classes, the tier ladder and the provider classification above, as a job specification a market can price against. The immediate consequence is that verification stops being a fixed protocol constant and becomes a per-job attribute chosen by whoever bears the loss — which is both cheaper in aggregate and safer at the tail, since no single setting is right for a chat completion and a settlement instruction simultaneously.

### 4.2 Confidentiality as a capability, not a promise

Four attestable tiers with local verification, and the workload they unlock: training and inference on weights the provider may not keep and data it may not copy. A model owner ships proprietary weights into an attested boundary, a data holder supplies a licensed corpus that never leaves it, and the provider is paid for accelerators without receiving either in extractable form.

This is the difference between a network that can serve public models on public data and one that can serve enterprise demand, and it is where the budgets are.

### 4.3 A post-quantum retrofit with no fork

Provider identity, session establishment, staking and receipt signatures are classical almost everywhere, which means recorded sessions are readable whenever a capable machine exists, and identity keys are forgeable from that moment forward. The retrofit is additive and wire-compatible: post-quantum material carried alongside classical rather than instead of it, so an old peer ignores a field it does not recognise and a new peer verifies both. Security becomes the maximum of the two assumptions rather than the minimum.

We have shipped this pattern twice — onto a mesh networking stack with a five-hundred-byte frame and a five-bit-per-second floor, and onto a defence-grade storage profile. A signed receipt grows by roughly three kilobytes; session establishment by about two, once per session. At market scale these are unremarkable against the model weights already in flight.

### 4.4 A market maker for heterogeneous capacity

Price as position and trend as momentum in a phase space, evolved by an energy-conserving integrator under a regime-dependent potential, with the regime recovered as the latent state of a hidden Markov model driven by request rate, queue depth and utilisation. Each resource kind carries its own market state and prices never cross between kinds. Per-tenant preference is a one-bit sign delta over a shared quote table, small enough to hold millions of tenant tables at once. Quotes are deterministic given a job and a market state, so a disputed quote is an arithmetic check rather than an argument.

This exists because a constant-product curve cannot express what compute is: perishable, heterogeneous, deadline-bound, and convex near exhaustion. The companion paper on the market mechanism treats it in full.

## 5 The composition

The four pieces are separable and each is adoptable alone, which is the property that makes them worth offering. Together they compose into a single path:

demand  $\rightarrow$  market maker prices it  $\rightarrow$  provider executes  $\rightarrow$  verification establishes what  
happened  $\rightarrow$  mint settles

Each arrow is a narrow interface. The market maker forms no opinion about honesty; verification forms none about price; the mint forms none about allocation. That separation is what allows any of them to be replaced without disturbing the others, and it is why adopting one does not commit a network to adopting the rest.

## 6 What this does not settle

Three things, stated because they will otherwise be discovered later.

Attributing model quality to individual data contributions remains open at scale; proportional payment weighted by measured improvement on held-out evaluation is crude, implementable, and not fair. Attested execution moves the trust root to a silicon vendor's certificate chain rather than removing it, and should be composed with sampled recomputation rather than relied on alone at high value. And the market maker's constants — friction, energy scale, potential shape per regime — are chosen and defensible rather than fit against a large live market, because the market that would fit them is what this proposes to build. The claim is that the shape is right for perishable heterogeneous capacity, not that the numbers are.